Software Requirements Specification Template

Software Engineering

The following annotated template shall be used to complete the Software Requirements Specification (SRS) assignment.

Template Usage:
Text contained within angle brackets (‘<’, ‘>’) shall be replaced by your project-specific information and/or details. For example, <Project Name> will be replaced with either ‘Smart Home’ or ‘Sensor Network’.

Italicized text is included to briefly annotate the purpose of each section within this template. This text should not appear in the final version of your submitted SRS.

This cover page is not a part of the final template and should be removed before your SRS is submitted.

Movie Theater Ticketing Service

Software Requirements Specification

Version 1.4

24 September 2026

Group #6
Liliana Yalda, Shayna Phan, Thi Dang

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Fall 2023

Revision History

Date	Description	Author	Comments
09-10-2026	First look	Liliana Yalda, Shayna Phan, Thi Dang	First look
09-17-2026	Added Intro, Purpose, and Scope material	Liliana Yalda, Shayna Phan, Thi Dang	
09-23-2026	Added section 2: General Description, and some of section 3: Specific requirements	Liliana Yalda, Shayna Phan, Thi Dang	
09-24-2026	Expansion of ideas and finalization/cleanup of document	Liliana Yalda, Shayna Phan, Thi Dang	

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	Liliana Yalda, Shayna Phan, Thi Dang	Software Eng.	09-24-2026
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

Revision History ii

Document Approval ii

1. Introduction 1

1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms, and Abbreviations 1
1.4 References 1
1.5 Overview 1

2. General Description 2

2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Characteristics 2
2.4 General Constraints 2
2.5 Assumptions and Dependencies 2

3. Specific Requirements 2

3.1 External Interface Requirements 3
3.1.1 User Interfaces *3*
3.1.2 Hardware Interfaces *3*
3.1.3 Software Interfaces *3*
3.1.4 Communications Interfaces *3*
3.2 Functional Requirements 3
3.2.1 <Functional Requirement or Feature #1> *3*
3.2.2 <Functional Requirement or Feature #2> *3*
3.3 Use Cases 3
3.3.1 Use Case #1 ..... *3*
3.3.2 Use Case #2 ..... *3*
3.4 Classes / Objects 3
3.4.1 <Class / Object #1> *3*
3.4.2 <Class / Object #2> *3*
3.5 Non-Functional Requirements 4
3.5.1 Performance *4*
3.5.2 Reliability *4*
3.5.3 Availability *4*
3.5.4 Security *4*
3.5.5 Maintainability *4*
3.5.6 Portability *4*
3.6 Inverse Requirements 4
3.7 Design Constraints 4
3.8 Logical Database Requirements 4
3.9 Other Requirements 4

4. Analysis Models 4

4.1 Sequence Diagrams 5
4.3 Data Flow Diagrams (DFD) 5
4.2 State-Transition Diagrams (STD) 5

5. Change Management Process 5

A. Appendices 5

A.1 Appendix 1 5
A.2 Appendix 2 5

1. Introduction

This Software Requirements Specification (SRS) document describes the requirements for a movie ticketing website. It defines the functionality, features, and constraints of the system and provides a detailed description of how the system is expected to operate.

1.1 Purpose

The purpose of this Software Requirements Specification is to document the requirements for the movie ticketing website. This document will serve as a guide for the software development team when designing, implementing, and testing the system. It is intended for the software developers, designers, testers, and other project members involved in developing the system.

1.2 Scope

The software product to be developed is a web-based movie ticketing system. The system will allow users to browse available movies at select locations, view movie information, search for available showtimes, select a showing, choose available seats, and purchase movie tickets through the website.

The system will provide users with an accessible way to view available movies and showtimes and complete the ticket purchasing process without needing to purchase tickets in person. The system will also manage information such as movies, theaters, showtimes, seats, and ticket purchases.

The primary goal of the system is to provide a simple and convenient method for users to find movie showtimes and purchase tickets. The system should accurately display available seats and prevent users from purchasing seats that are no longer available.

The system will focus on online movie ticketing and will not be responsible for functions outside of the ticketing system, such as physically operating a movie theater, managing theater employees, or controlling movie playback.

1.3 Definitions, Acronyms, and Abbreviations

1.3.1 E-Tickets: A digital ticket allows customers to enter a movie showing.

1.3.2 Seat Hold: The selected tickets/seats will be held for 5 minutes while the customer completes the purchase.

1.3.3 Refund Policy: Tickets can be returned or exchanged prior to the showing only if purchased with a user account

1.3.4 External Payment: Accepted payment methods include credit card, PayPal, and bitcoin.

1.3.5 UC: Use Case

1.3.6 FR: Functional Requirement

1.4 References

IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications

Massimo Felici-School of Informatics 2011, Use Cases

William Y. Arms-Cornell University Computing and Information Science, CS 5150 Software Engineering Scenarios and Use Cases

1.5 Overview

This SRS is organized into five major sections. Section 1 introduces the purpose, scope, terminology, references, and organization of the document. Section 2 provides a general description of the movie ticketing system, including its perspective, major functions, users, constraints, assumptions, and dependencies. Section 3 defines the specific functional and non-functional requirements of the system, including external interfaces, use cases, classes and objects, performance, reliability, availability, security, maintainability, and portability.

2. General Description

2.1 Product Perspective

The movie ticketing system is a web-based application that will interact with several external and internal services to provide online ticket purchasing and ticket management.

2.1.1 System Interfaces

The movie ticketing system will need to handle payments. Thus, connection to banking interfaces and a payment gateway will be necessary in order to process transactions, including payments and refunds. This will connect to services such as PayPal, Apple Pay, and bank/card processors.

The system will also need to connect to internal interfaces for theater management in order to have access to movie information, showtimes, theater locations, ratings, descriptions, and other details about the films. In a similar fashion, the site will need access to showroom information, such as seat layout and real-time availability. This information will also be available to a management interface for logistics such as sales and attendance.

The system will also need integration with an Email/SMS notification service, which will send booking confirmations, electronic tickets, cancellation notices, and other notifications to customers. With this, a login will be necessary, so the system will integrate with authentication

and identity verification such as Google or Apple for login information, as well as a database of customers and accounts and potential membership status within the cinema's loyalty program. Authentication will also need to recognize and block access for any bots trying to mass purchase tickets.

2.1.2 User Interfaces

The system will need to be well organized, straightforward, and easy to use for a wide range of customers. This will require design choices that prioritize readability, such as simple fonts and high-contrast colors, as well as clearly labeled buttons, as customers such that new customers who have never used the system before can easily purchase tickets in under 5 minutes. If an error occurs, it will be properly displayed such that the customer can understand it without prior knowledge, and provide resources for further assistance.

2.2 Product Functions

The ticketing software will allow customers and employees to create accounts and log in. This will enable them to view previous purchases, handle returns and exchanges, manage membership and loyalty program information, and initiate new purchases.

When purchasing a ticket, the system will provide a real-time view of the available theaters, rooms, and showtimes for each movie, as well as the seat availability and layout of the chosen location. After choosing seats, there will be a hold for 5 minutes to allow the user to purchase a maximum of 20 tickets at a time, calculating and displaying the running total plus tax.

Users will then be prompted to provide payment and contact information, and order confirmation and E-tickets will be sent to their preferred email once the purchase is verified via communication with the banking interface. Within their account, customers will be able to manage tickets, including refunds, exchanges, and cancellations, which will update seat availability for other customers.

2.3 User Characteristics

Customers will be those using the software to purchase and manage their tickets and view showtimes. For the sake of usability, customers are assumed to have basic experience with web browsers and nothing beyond common knowledge.

Employees will have access to the system to check and validate purchases and manage returns and exchanges. Employees should be able to learn the system with some basic training.

Administrators will be able to manage accounts and membership information, as well as troubleshoot more complicated system management, including settings and security. They have more experience and in-depth knowledge of how such systems are expected to work.

Managers can view sales and attendance reports and manage showtime schedules and pricing. They are experienced in management and familiar with such processes.

2.4 General Constraints

There are some general constraints for the Movie Theater Ticketing Service. The system needs to keep customer and payment information safe. It should work on common web browsers and mobile devices. The system also needs a database to store information about movies, customers, tickets, and seats. For online payments, the system may need to use an outside payment service. The system should be easy to use for customers and employees. It should also be possible to update and maintain the system in the future.

2.5 Assumptions and Dependencies

We assume that customers will have access to the internet when they use the system online. We also assume that the theater will have computers or other devices for employees to use the system. The system depends on a database to store and access movie, customer, seat, and ticket information. Online payment also depends on an external payment service working properly. If the internet, database, or payment service is not available, some functions of the system may not work correctly. Changes to these factors may require changes to the system requirements.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The system will be used by opening a web browser on a computer, a tablet, or a phone. Customers will be able to look for movies, check showtimes, see how much tickets cost, choose seats when they are available, and buy tickets. The system will also have pages for logging in, signing up, having a loyalty account, making payments, getting ticket details, and giving customer feedback.

The system will work with English, Spanish, and Swedish. The way the system looks should be simple and easy to understand. Administrators and employees will have screens and options for handling theaters, movies, showtimes, tickets, and customer details.

3.1.2 Hardware Interfaces

The system does not require special hardware because it will run through a web browser. Customers can use a computer, tablet, or smartphone with stable internet access. Theater employees can use computers or other supported devices to access the system.

The system may also work with theater hardware such as ticket scanners or barcode/QR code scanners to verify digital tickets. The hardware must be able to connect to the system so ticket information can be checked.

3.1.3 Software Interfaces

The system will connect with several other software systems. It will connect to a database to store customer, movie, showing, ticket, payment, and purchase information. The system will also connect with payment services such as credit cards, PayPal, and Bitcoin.

The system may connect to external movie review websites such as Rotten Tomatoes or IMDB to get movie ratings and critic information. It may also connect with currency exchange services to convert ticket prices to the local currency. The system will need to work with these external services when they are available.

3.1.4 Communications Interfaces

The system will use the internet to communicate between customers, employees, the web server, and the database. Communication should use secure connections to protect customer and payment information.

The system will need to handle many users at the same time and provide live updates for ticket availability and showtimes. When there are too many requests for a high-demand showing, the system will use a queue to control the number of requests. The system should also communicate with payment services and other external services through secure network connections.

3.2 Functional Requirements

3.2.1 Ticket Purchase

3.2.1.1 Introduction

The ticket purchase feature allows customers to select a movie, choose a showing, select tickets or seats, and complete a purchase online. Customers can purchase up to 20 tickets in one transaction. Tickets are available starting two weeks before the showing.

3.2.1.2 Inputs

The system will receive the following inputs from the customer: selected movie, showtime, number of tickets, selected seats for deluxe theaters, ticket type, payment method, and payment information

3.2.1.3 Processing

The system will check the selected movie and showtime and then check the available tickets. The system will calculate the ticket price based on the ticket type and any discount. The system will also make sure the customer is not buying more than 20 tickets at a time. The selected tickets will be held for 5 minutes while the customer completes the purchase.

The system will process the payment and, if the payment is successful, create E-tickets. The system will update the available tickets and save the purchase in the database.

3.2.1.4 Outputs

After a successful purchase, the system will display a purchase confirmation. The customer will receive their E-tickets and payment information. The system will also update the available tickets, purchase history, and loyalty points if the customer has a loyalty account.

3.2.1.5 Error Handling

If there are not enough tickets available, the system will tell the customer that the tickets are unavailable. If the customer tries to purchase more than 20 tickets at a time, the system will not allow the purchase. If payment fails, the system will show an error, and the customer can try again. If the 5-minute purchase time expires, the selected tickets will be released so other customers can purchase them. If there are too many users trying to purchase tickets for the same showing, the customer will be placed in a queue.

3.2.2 User Account Management

3.2.2.1 User Registration

The system will allow the user to create an account by providing a valid name, email address, and password. A verification code/link will be sent to the email upon initial sign-up.

3.2.2.2 Input Validation

The system will reject requests when a required field is left empty or in an invalid format, such as if the password does not match security requirements. Duplicate accounts will also be prevented if the provided email is already associated with an existing account.

3.2.2.3 Authentication

Users will be authenticated via a secure check between the entered email and password and the registered information in the database and verified as a human via CAPTCHA.
`

3.2.3 Movie Search and Selection

3.2.3.1 Movie Search

The system shall accept a movie title or keyword as input and return a display of available movies matching the provided search criteria. An appropriate message will appear if no movies match the search criteria.

3.2.3.2 Movie Information

The system will display the movie's title, description, genre, duration, rating, release date, language, and available formats. It will indicate whether a movie is currently showing, scheduled for a future date, or unavailable for booking.

3.2.4 Seat Selection

3.2.4.1 Seat Layout

The system will display the seating layout associated with the selected showroom, along with row letters, seat numbers, the screen and walkways in the room, and handicapped seats.

3.2.4.2 Seat Availability

The system will identify and display available seats, temporarily reserved seats, and unavailable or taken seats. One or more seats can be selected at a time. Users will be prevented from selecting seats that are already reserved, in the process of being reserved, or unavailable.

3.2.4.3 Concurrent Booking

The software must verify seat availability before confirming a booking to prevent two users from purchasing the same seat.

3.2.4.4 Seat Reservation Timeout

The system shall mark seats as temporarily reserved for 5 minutes while the user engages in the payment and verification process. If the payment is abandoned or exceeds the configured time, the system will release the seats and make them available again.

3.3 Use Cases

3.3.1 Use Case #1

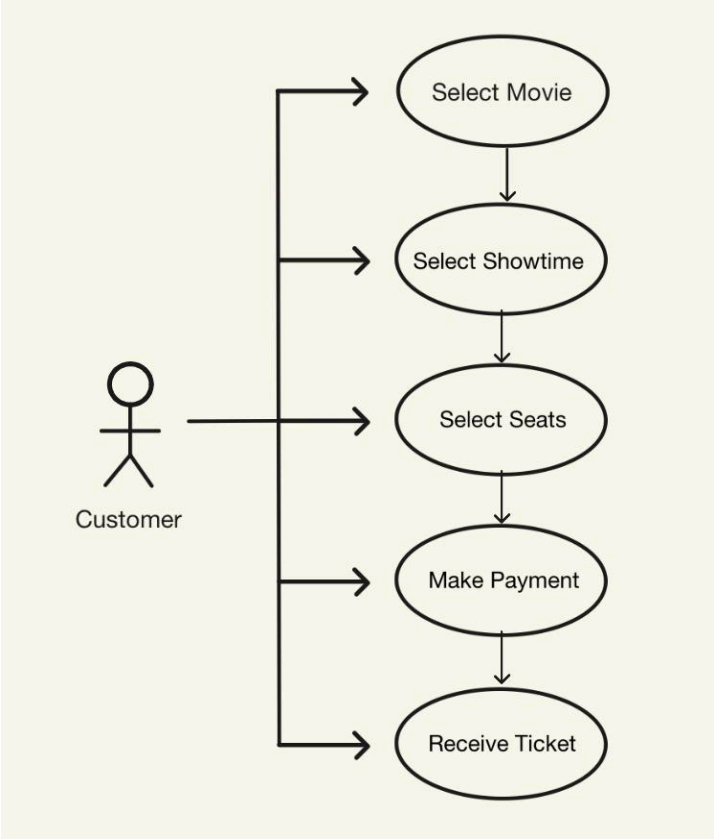
Book Movie Ticket

Actor(s): Customer

Flow of Events:

1. Customer searches for and selects a movie
2. Customer selects a cinema, date, and showtime
3. Customer selects type of ticket and applies applicable discount
4. System displays the available seats
5. Customer selects one or more seats
6. System verifies that the selected seats are available
7. System calculates the total ticket price
8. System displays the booking summary
9. Customer confirms the booking
10. System processes payment
11. If payment is successful, the system confirms the booking

12. System generates an electronic ticket
13. System displays/sends the e-ticket to the customer



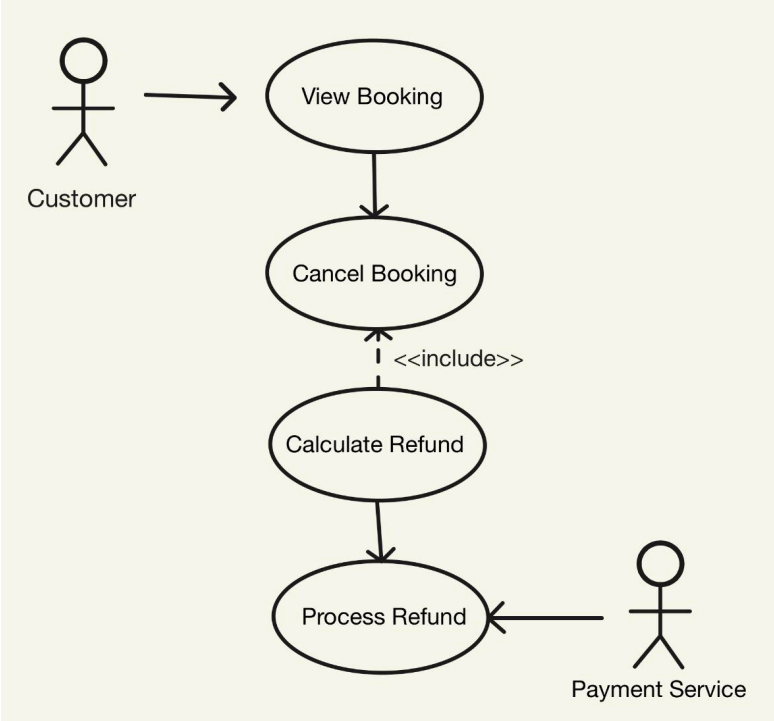
3.3.2 Use Case #2

Cancel Booking

Actor(s): Customer, Payment Service

Flow of Events:

1. Customer logs into their account
2. Customer views their existing bookings
3. Customer selects the booking they wish to cancel
4. System retrieves the booking information
5. System checks whether the booking is eligible for cancellation
6. System displays the applicable cancellation and refund information
7. Customer confirms the cancellation
8. System changes the booking status to canceled
9. System releases the reserved seats
10. System calculates the applicable refund
11. System sends the refund request to the Payment Service
12. Payment Service processes the refund
13. System displays the cancellation and refund status to the customer



3.3.3 Use Case #3
Redeem Ticket

Actor(s): Cinema Staff

Flow of Events:
1. Cinema Staff uses provided hardware to scan ticket/rewards
2. Software verifies ticket/rewards
3. Rewards are applied according to the loyalty program
4. Database validates entry
5. System marks ticket as used

3.4 Classes / Objects

3.4.1 User Roles

3.4.1.1 Attributes:
- Role: Customer, Manager, and Employee
- Account
  -User ID
  - Name
  - Email address
  - Password

- Membership information

3.4.1.2 Functions:

- Set Role Privileges
- Change Role
- Create account
- Log in
- Manage account
- View purchase history
- View loyalty information

3.4.2 Tickets

3.4.2.1 Attributes:

- Redeemed or Not
- Seat assigned
- Showroom Assigned
- Customer who owns the ticket
- Type of ticket: Adult, Child, Senior, Military, Student

3.4.2.2 Functions:

- Purchase/Assign Ticket to Customer
- Check-in Ticket
- Refund Ticket
- Cancel Ticket

3.4.3 Seats

3.4.3.1 Attributes:

- Availability: Available → Held → Sold
- Ticket associated with Seat
- Row Letter
- Seat number
- Regular versus deluxe seat

3.4.3.2 Functions:

- Change Availability
- Hold seat
- Release seat

3.4.4 Auditorium

3.4.4.1 Attributes:

- Seat Map and Locations

- Capacity
- Accessibility Constraints
- Movie to be Screened
- Deluxe versus regular showroom

3.4.4.2 Functions:

- Create/Edit Auditorium
- Update Seat
- Update Availability
- Update Movie

3.4.5 Payment

3.4.5.1 Attributes:

- Status: Authorized → Captured/Refunded/Failed
- Payment ID
- Payment method
- Associated purchase

3.4.5.2 Functions:

- Check Payment Information
- Validate with Payment Service
- Submit payment to payment service
- Process refund

3.4.6 Policy Values

3.4.6.1 Attributes:

- Seat-Hold Duration
- Refund Window
- Orphan Seat Requirements
- Maximum Tickets per Transaction

3.4.6.2 Functions:

- Configure Policy Values

3.4.7 Screening

3.4.7.1 Attributes:

- Screening ID
- Movie
- Auditorium
- Date and Time

- Publication State: Draft → Published → Unpublished/Cancelled

3.4.7.2 Functions:

- Create Screening
- Update Screening
- Update Publication State
- Cancel Screening

3.5 Non-Functional Requirements

3.5.1 Performance

The system shall meet the following performance requirements under normal operating conditions:

3.5.1.1 Quick to load: 95% of loads complete within 5 seconds.

*3.5.1.2 Quick to transac*t: 95% of transactions complete within 8 seconds.

3.5.1.3 Quick to update: 99% of user interactions with the webpage should register within 3 seconds.

3.5.1.4 Bandwidth: The server can handle 1000 simultaneous requests at once.

3.5.2 Reliability

The system must handle any errors that will cripple the service.

3.5.2.1 Error Handling: The system will be able to identify and, in certain cases, correct errors.

3.5.3 Availability

The service will be available as often as possible, allowing for occasional maintenance periods.

3.5.3.1 Consistent Uptime: The server and website are available for public use 99.9% of the time.

3.5.4 Security

The service will protect customers' data and prevent unauthorized users from accessing and modifying sensitive data.

3.5.4.1 Authorization: only managers or higher-level clearances shall be able to access secure information from databases.

3.5.4.2 Encryption: customer data should be encrypted with current RSA encryption.

3.5.5 Maintainability

The software will be designed to last and work without frequent and drastic revision upon release. There will be small patches to cover unforeseen problems with the system on a continual basis.

3.5.5.1 Lifespan: The software should be able to function over the next 3 years with minor revisions.

3.5.5.2 Frequency of Revisions: Minor updates will occur every 3 months, while major changes will occur every 3 years.

3.5.6 Portability

The website should run without errors or problems in the most frequently used browser engines.

3.5.6.1 *Chrome*: Designed for the Chrome browser

3.5.6.2 Common Compatibility: Runs without errors in the 3 most common browsers: Edge, Safari, and Firefox

3.6 Inverse Requirements

*State any *useful* inverse requirements.*

3.7 Design Constraints

Specify design constrains imposed by other standards, company policies, hardware limitation, etc. that will impact this software project.

3.8 Logical Database Requirements

Will a database be used? If so, what logical requirements exist for data formats, storage capabilities, data retention, data integrity, etc.

3.9 Other Requirements

Catchall section for any additional requirements.

4. Analysis Models

List all analysis models used in developing specific requirements previously given in this SRS. Each model should include an introduction and a narrative description. Furthermore, each model should be traceable the SRS's requirements.

4.1 Sequence Diagrams

4.3 Data Flow Diagrams (DFD)

4.2 State-Transition Diagrams (STD)

5. Software Design

5.1 Software Architecture Overview

5.1.1 Presentation Layer

This is the part that users interact with through a web browser. Customers can search movies, view showtimes, select seats, purchase tickets, and manage their accounts. Employees, managers, and administrators have different interfaces based on their roles. The system can be accessed through computers, tablets, and smartphones.

5.1.2 Application Layer

This layer contains the main business logic of the system. It handles user accounts, movie searches, screenings, seat selection, ticket purchases, payments, refunds, loyalty information, and policies. It also handles the 5-minute seat hold and a maximum of 20 tickets per transaction.

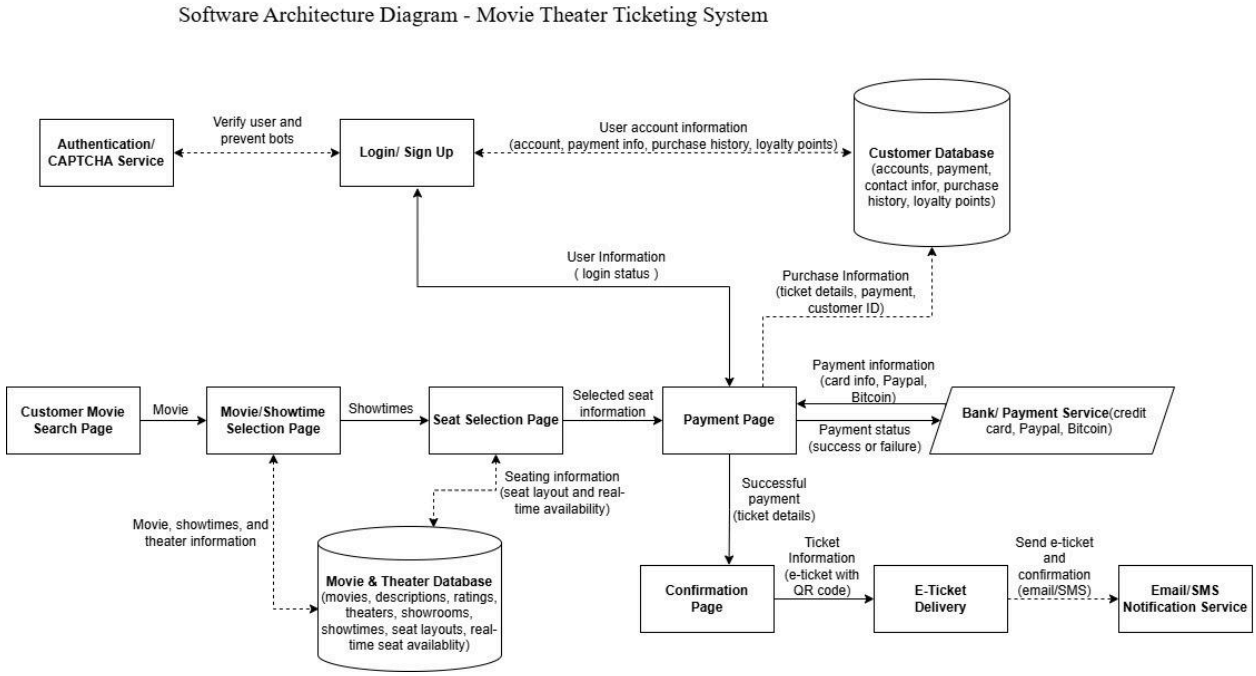
5.1.3 Data Layer

The database stores users, movies, screenings, seats, tickets, payments, and purchase information. The application layer reads and updates this information when customers make purchases or when employees and managers update the system.

5.1.4 External Services

The system communicates with external services for payment processing, email/SMS notifications, authentication, CAPTCHA/bot protection, movie reviews, and currency conversion. These services allow the movie ticketing system to provide functions that are outside the main application.

5.2 Architectural Diagram



5.2.1 Customer Movie Search Page

The Customer Movie Search Page allows the customer to enter a movie or keyword and view available movies. The system retrieves movie and theater information from the Movie & Theater Database.

The database contains information such as movie titles, descriptions, ratings, theaters, showrooms, and available showtimes.

5.2.2 Movie/Showtime Selection Page

After selecting a movie, the customer goes to the Movie/Showtime Selection Page. This page displays available theaters, dates, and showtimes for the selected movie.
The page receives movie, theater, and showtime information from the Movie & Theater Database. The customer then selects the showing.

5.2.3 Seat Selection Page

After selecting a showtime, the customer is taken to the Seat Selection Page. This page displays the seating layout and current seat availability for the selected auditorium.
The system obtains seating information from the Movie & Theater Database. Seats can be available, temporarily held, or unavailable/sold. When a customer selects a seat, the system verifies that the seat is still available before continuing with the purchase.
Selected seats are held for 5 minutes while the customer completes the payment process. If the customer does not complete the purchase within the time limit, the seats are released.

5.2.4 Login / Sign Up Page

The Login / Sign Up Page allows customers to create an account or log into an existing account. The system stores customer information in the Customer Database, including account information, contact information, purchase history, and loyalty information. New accounts require a valid name, email address, and password. The system also prevents duplicate accounts and validates login information.
The architecture also connects authentication with CAPTCHA/bot protection

5.2.5 Payment Page

The Payment Page receives the customer's selected seat information and allows the customer to provide payment information.

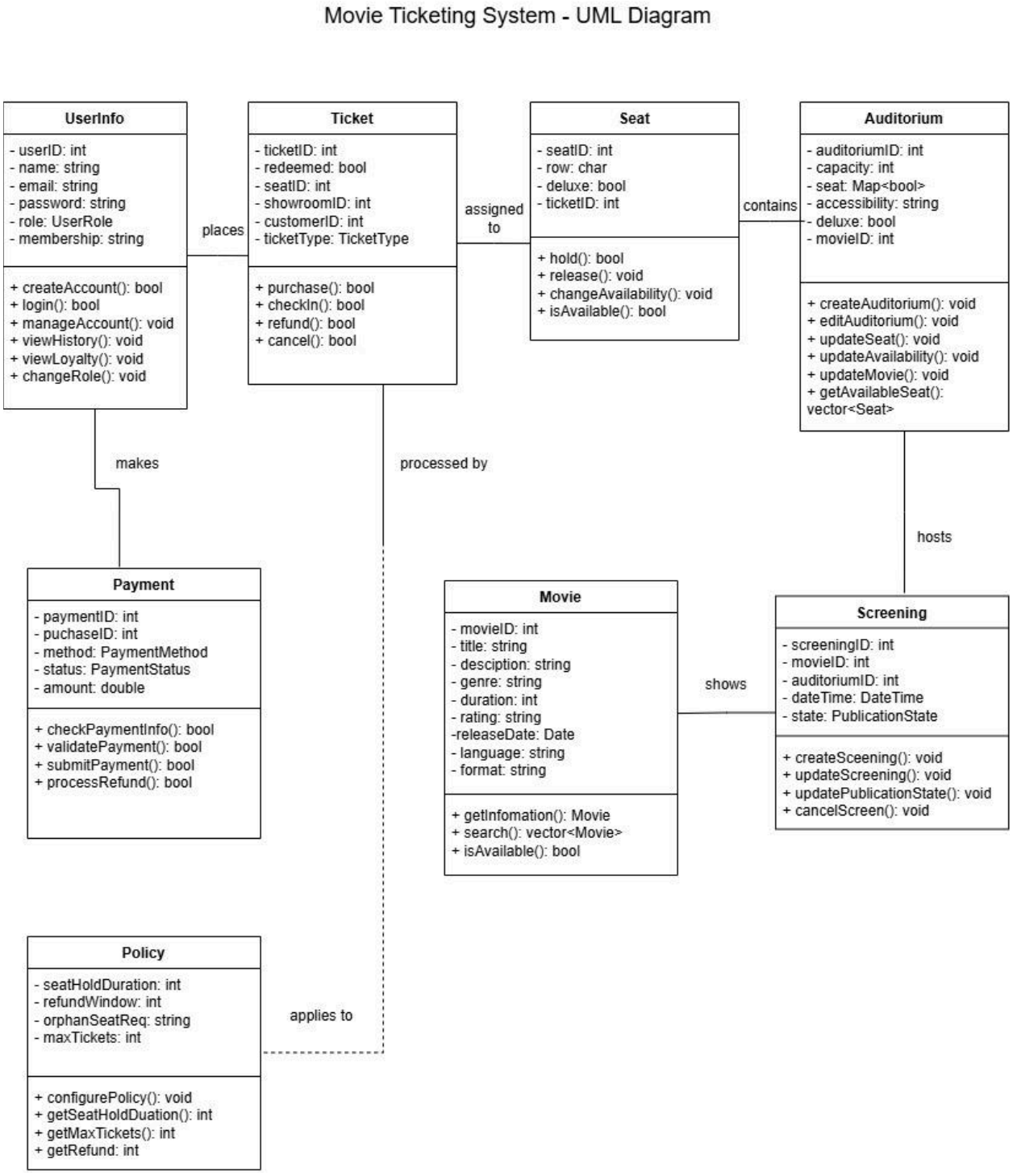
5.3 UML Class Diagram

The following classes represent the primary objects required by the Movie Theater Ticketing System:

- UserInfo
- Movie
- Screening
- Auditorium
- Seat
- Ticket

- Payment
- Policy

The external Payment Service is represented as a service interface rather than as a database entity.



***Changes to the Software Architecture Overview**

The software architecture overview from Assignment 2 remains unchanged in Assignment 3. The existing architecture, consisting of the presentation layer, application layer, data layer, and external services, continues to support the requirements of the Movie Theater Ticketing System. The features covered by the new test plan, including movie searches, seat reservations, payment processing, and ticket creation, can be tested using the existing architecture and components. Therefore, no architectural changes were necessary for this assignment. The updates in Assignment 3 focus on adding a test plan to verify that the system's components function correctly both individually and together.

5.4 Class Descriptions

5.4.1 UserInfo

The UserInfo class represents a registered customer, employee, or manager. It stores authentication and account information and provides operations for account management and authentication.

5.4.2 Movie

The Movie class represents a movie available or scheduled to be shown by the theater. It contains information such as the title, genre, duration, rating, and release date.

5.4.3 Screening

The Screening class represents a scheduled showing of a movie in a specific auditorium. It contains the movie, auditorium, date, time, and publication state.

5.4.4 Auditorium

The Auditorium class represents a theater room in which screenings occur. It maintains information about capacity, accessibility requirements, theater type, and the seats contained within the auditorium.

5.4.5 Seat

The Seat class represents an individual seat within an auditorium. A seat can be available, temporarily held, or sold. The class is responsible for changing the seat’s availability and enforcing the temporary reservation period.

5.4.6 Ticket

The Ticket class represents an individual electronic ticket associated with a movie, screening, and seat. Tickets may be redeemed, canceled, or refunded according to the system’s policies.

5.4.7 Payment

The payment class represents a financial transaction associated with a booking. It stores payment information and tracks whether the payment was authorized, captured, refunded, or failed.

5.4.8 Policy

The Policy class stores configurable business rules used throughout the system. Examples include the five-minute seat-hold duration, refund window, and maximum number of tickets allowed per transaction.

5.5 Attribute Descriptions

Class	Attribute	Data Type	Description
UserInfo	userID	int	Unique identifier for the user
UserInfo	name	string	User's full name
UserInfo	email	string	User's registered email address
UserInfo	password	string	Securely stored password representation
UserInfo	role	userRole	Identifies customer, employee, or manager privileges
UserInfo	membership	string	Identifies user's level of membership
Ticket	ticketID	int	Unique identifier for a ticket
Ticket	redeemed	bool	Indicates whether the ticket has been redeemed
Ticket	seatID	int	Identifier for assigned seat
Ticket	showroomID	int	Identifier for showroom
Ticket	customerID	int	Unique identifier that connects ticket to customer
Ticket	ticketType	ticketType	Adult, child, senior, military, or student
Seat	seatID	int	Unique identifier for a seat
Seat	row	char	Letter identifying seat row
Seat	deluxe	bool	Indicates whether seat is deluxe
Seat	ticketID	int	Ticket associated with seat
Auditorium	auditoriumID	int	Unique identifier for an

			auditorium
Auditorium	capacity	int	Maximum number of seats
Auditorium	seat	Map<bool>	Map vector of seating arrangement

Auditorium	accessibility	string	Accessibility information and constraints
Auditorium	deluxe	bool	Indicates whether auditorium has deluxe seating
Auditorium	movieID	int	Identifies movie playing in auditorium
Payment	paymentID	int	Unique identifier for a payment
Payment	purchaseID	int	Identifies the transaction of the payment
Payment	method	PaymentMethod	Indicates method used for payment
Payment	status	PaymentStatus	Pending, paid, refunded, failed
Payment	amount	double	Amount charged or refunded
Movie	movieID	int	Unique identifier for a movie
Movie	title	string	Title of the movie
Movie	description	string	Short description of the movie
Movie	genre	string	Genre of the movie
Movie	duration	int	Length of the movie
Movie	rating	string	Rating of the movie
Movie	releaseDate	Date	Date movie was released

Movie	language	string	Screening language of the movie
Movie	format	string	Movie format
Screening	screeningID	int	Unique identifier for the screening
Screening	movieID	int	Movie associated with the screening
Screening	auditoriumID	int	Auditorium associated with the screening
Screening	dateTime	DateTime	Date of the screening
Screening	state	PublicationState	Indicates whether the screening is draft, published, or cancelled
Policy	seatHoldDuration	int	Number of minutes a seat can remain held
Policy	refundWindow	int	Maximum period for eligible refunds
Policy	orphanSeatRequirement	string	Avoids orphan seating
Policy	maxTickets	int	Maximum seats permitted per transaction

5.6 Operation Descriptions

Class	Operations	Return Type	Parameters	Description
UserInfo	createAccount()	bool	string name, string email, string password	Create an account
UserInfo	login()	bool	string email, string password	Verifies the user's credentials
UserInfo	manageAccount()	void	none	Updates account information
UserInfo	viewHistory()	void	none	Returns the user's

				previous bookings
UserInfo	viewLoyalty()	void	none	Returns the user's loyalty information
UserInfo	changeRole()	void	userRole newRole	Changes the role of the user
Ticket	purchase()	bool	int customerID, int screeningID	Purchases a ticket
Ticket	checkIn()	bool	int customerID	Marks a ticket as redeemed
Ticket	refund()	bool	none	Refunds a ticket
Ticket	cancel()	bool	none	Cancels a ticket
Seat	hold()	bool	int userID, int durationMinutes	Hold the seat in duration time
Seat	release()	void	none	Releases a held seat
Seat	changeAvailability()	void	SeatStatus newStatus	Changes the seat's availability
Seat	isAvailable()	bool	none	Checks whether the seat is available
Auditorium	createAuditorium()	void	int capacity, bool deluxe	Create auditorium with capacity and type
Auditorium	editAuditorium()	void	none	Modify auditorium's information
Auditorium	updateSeat()	void	int seatID, SeatStatus status	Updates seat status

Auditorium	updateAvailability()	void	none	Update the availability information for the auditorium's seats
Auditorium	updateMovie()	void	int movieID	Updates the movie to be played in the auditorium

Auditorium	getAvailableSeat()	vector<Seat>	none	Returns all available seats
Payment	checkPaymentInfo()	bool	int	Check if the payment information complete and valid
Payment	validatePayment()	bool	int	Checks whether a payment is valid from external payment service
Payment	submitPayment()	bool	double amount	Submit payment for external payment service
Payment	processRefund()	bool	double amount	Check if the payment eligible to get refund
Movie	getInformation()	Movie	int movieID	Returns movie information
Movie	search()	vector<Movie>	string keyword	Searches for a movie with provided keywords
Movie	isAvailable()	bool	Date date	Check if the movie available for the specific date
Screening	createScreening()	void	movieId: int, auditoriumId: int, dateTime: DateTime	Creates a new screening
Screening	updateScreening()	void	dateTime: DateTime	Updates screening information
Screening	updatePublicationState()	void	none	Publishes a screening so it can be viewed/booked
Screening	cancelScreening()	void	none	Cancels a screening
Policy	configurePolicy()	void	holdDuration: int, refundWindow: int, maxTickets:	Updates configurable system policy values

			int	
Policy	getSeatHoldDuration()	int	none	Returns the configured seat-hold duration
Policy	getMaxTickets()	int	none	Returns the maximum number of tickets permitted
Policy	getRefund()	int	amount: decimal	Requests a refund for the specified amount

5.7 Development Plan and Timeline

5.7 Development Plan and Timeline

The development of the Movie Theater Ticketing System will be divided among the three team members. Each member will have primary responsibility for specific portions of the design while all team members will participate in review and integration.

5.7.1 Task Partitioning

The major development tasks are:

1. Define and document the software architecture.
2. Create the architectural diagram.
3. Identify and organize the system classes.
4. Create the UML class diagram and relationships.
5. Document class attributes and data types.
6. Document class operations, parameters, and return types.
7. Review the design against the existing functional requirements.
8. Combine all sections into the SRS document.
9. Review and proofread the completed document.
10. Upload the completed project materials to the team's GitHub repository.

5.7.2 Team Member Responsibilities

Team Member	Primary Responsibilities
Lili	Software architecture overview, architectural diagram, integration of design sections into the SRS

Shayna	UML class diagram, class relationships, verification that classes correspond to system requirements
Thi	Class descriptions, attribute descriptions, operation descriptions, data types and interfaces
Lili, Shayna, and Thi	Final review, editing, requirements/design consistency check, and GitHub submission

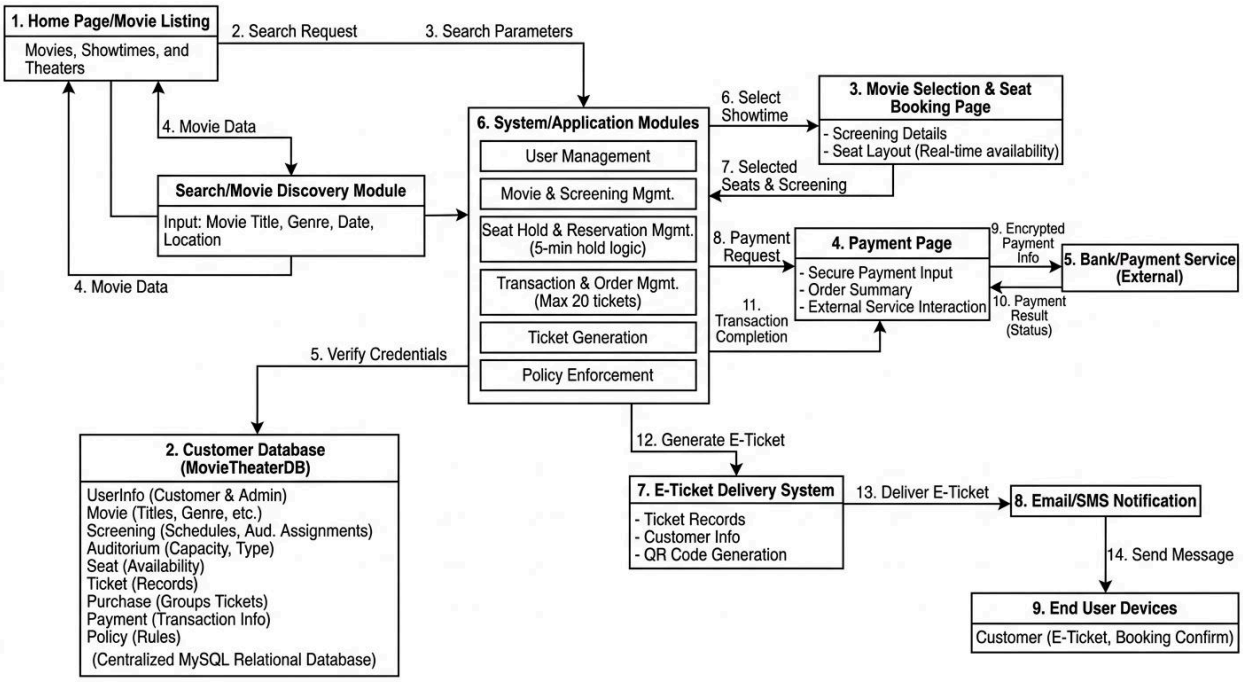
5.7.3 Development Timeline

Phase	Task	Responsible Member(s)
1	Review previous SRS requirements	Lili, Shayna, Thi
2	Define software architecture	Lili
3	Create architectural diagram	Lili
4	Identify and organize UML classes	Shayna
5	Create UML class diagram	Shayna
6	Document class attributes	Thi

7	Document operations and interfaces	Thi
8	Review UML and architecture against requirements	Lili, Shayna, Thi
9	Combine all Design sections into the SRS	Lili
10	Final proofreading and corrections	Lili, Shayna, Thi
11	Upload final SRS and supporting files to GitHub	Lili, Shayna, Thi

The design will be reviewed by all team members before submission to ensure that the architecture, UML model, class descriptions, and operations remain consistent with the previously defined functional requirements.

5.8 Updated Software Architecture Diagram



The existing software architecture diagram has been updated to include the selected database technology and its connection to the system's application layer. The system will use a centralized MySQL relational database named MovieTheaterDB to store and manage persistent information. The updated diagram also identifies the external Payment Service and illustrates how the application layer communicates with both the database and external services.

Modifications from Assignment 2

Compared to the original architecture from Assignment 2, the overall layered structure remains unchanged. The main modification is the expansion of the Data Layer to specify the database technology and storage organization. MySQL was selected as the database management system, with a centralized database named MovieTheaterDB. The updated diagram also makes the connection between the application layer, database, and external Payment Service more explicit. These modifications provide a more detailed representation of how the system manages and accesses persistent data.

5.9 Data Management Strategy

5.9.1 Database Technology Selection

The Movie Theater Ticketing System will use MySQL as its database management system. MySQL is a relational SQL database that organizes information into tables and establishes relationships using primary keys and foreign keys.
A relational database was selected because the system contains multiple entities that are connected to one another. For example, customers can purchase multiple tickets, movies can have multiple screenings, and each screening is associated with an auditorium. MySQL supports

these relationships and provides transaction management, which is important for maintaining consistency during ticket purchases and seat reservations.

5.9.2 Number of Databases

The system will use one centralized database named MovieTheaterDB. A single database was selected because the system is relatively small and its information is closely related. Using multiple databases would introduce additional complexity without providing significant benefits for the current project.

5.9.3 Database Organization

MovieTheaterDB will contain separate tables corresponding to the system's main entities.

Entity	Description
UserInfo	Stores customer and administrator account information.
Movie	Stores movie information, including title, genre, duration, rating, and release date.
Screening	Stores movie screening schedules and auditorium assignments.
Auditorium	Stores theater room information, including capacity and type.
Seat	Stores individual seat information and availability.
Ticket	Stores ticket records, including customer, seat, and screening associations.
Purchase	Stores customer purchase transactions and groups tickets belonging to a purchase.
Payment	Stores payment transaction information, including payment ID, purchase ID, amount, method, and status.
Policy	Stores system rules, including seat-hold duration, refund windows, and maximum ticket limits.

The tables will be connected through primary and foreign keys to maintain relationships between records. For example, a screening will reference a movie and auditorium, while tickets will reference the customer, screening, and seat associated with the purchase.

5.9.4 Data Integrity and Consistency

The system will use primary keys, foreign keys, and database constraints to maintain data accuracy and prevent invalid records. Transactions will be used for important operations such as ticket purchases. When a purchase is completed, the system must consistently record the purchase, create the ticket, update the seat's availability, and record the payment result.
The system will also enforce its configured policies, including the five-minute seat-hold duration and the maximum of 20 tickets per transaction.

5.9.5 Data Security and Privacy

Access to the database will be restricted to authorized application components and administrators. Customer passwords will be stored using secure password hashing rather than plain text. Secure connections will be used when communicating with the database where supported.
The external Payment Service will handle sensitive payment card information. The system will not store full payment card details in MovieTheaterDB. Instead, it will store the transaction

information needed for its records, such as payment identifiers, purchase identifiers, amounts, and payment statuses.

5.9.6 Data Backup and Recovery

The database should be backed up regularly to reduce the risk of losing important information. Backups will include user accounts, movie information, screening schedules, ticket records, and payment transaction records. Backup files should be stored securely and used to restore the database if data loss or system failure occurs.

5.10 Alternative Technologies and Design Tradeoffs

5.10.1 Relational Database vs. NoSQL Database

An alternative to MySQL would be a NoSQL database, such as MongoDB. NoSQL databases provide flexible document-based storage and can be useful for applications with large amounts of unstructured or frequently changing information.
However, the Movie Theater Ticketing System primarily manages structured information with clearly defined relationships. MySQL was selected because relational tables and foreign keys provide a suitable way to represent relationships between customers, screenings, seats, tickets, and payments.

5.10.2 One Database vs. Multiple Databases

Another alternative would be to separate customer information, movie information, and payment records into multiple databases. This could provide greater separation between different types of information and could be useful for a larger system.
However, multiple databases would increase complexity and require additional coordination when retrieving or updating related information. A single centralized database was selected because the system is relatively small and its data is closely connected.

5.10.3 Other SQL Alternatives

Other relational database management systems, such as PostgreSQL and Microsoft SQL Server, could also support the system's requirements. Both provide relational database functionality and support data integrity. MySQL was selected as a practical option that provides the functionality needed for this project.

5.10.4 Design Tradeoffs

The selected database strategy provides several advantages:

- Centralized storage and management of system information.
- Clear relationships between database tables.
- Transaction support for ticket purchases and seat reservations.
- Simplified database management.
- Improved data consistency through primary and foreign keys.

However, this approach also has some limitations:

- A single database may become a performance limitation as the system grows.
- The system depends on the availability of the centralized database.
- Changes to the relational structure may require updates to existing tables and relationships.

- Additional scaling and backup strategies may be needed as the amount of stored information increases.

Despite these limitations, a single MySQL relational database is appropriate for the current size and requirements of the Movie Theater Ticketing System.

6. Test Plan

6.1 Introduction

The purpose of this test plan is to verify that the Movie Theater Ticketing System meets its functional requirements and that its components work together correctly. Testing will focus on movie searches, user accounts, seat availability, temporary seat holds, payment processing, and ticket purchases.

The test plan includes unit, functional, integration, and system testing. Ten test cases are defined to evaluate normal system operations and possible failure conditions. The tests are based on the software design described in Section 5.

***Note:** For information on the updated Software Architecture Overview and diagrams, refer to the asterisk in Section 5.3.

6.2 Features to Be Tested

The following features will be tested:

- **Movie Search:** Verify that customers can search for movies and retrieve matching results.
- **User Accounts:** Verify that users can create accounts and that duplicate email addresses are rejected.
- **Seat Availability:** Verify that the system correctly identifies available, held, and sold seats.
- **Seat Holds:** Verify that selected seats remain held for five minutes and become available again when the hold expires.
- **Payment Processing:** Verify that payments are validated and that failed payments do not result in completed purchases.
- **Ticket Purchases:** Verify that successful payments result in ticket creation and the correct seat status.
- **Purchase Limits:** Verify that the system enforces the maximum of 20 tickets per transaction.
- **Concurrent Purchases:** Verify that two customers cannot purchase the same seat for the same screening.

6.3 Testing Methods

6.3.1 Unit Testing

Unit testing will verify individual methods and classes independently. It will focus on the accuracy of movie searches, policy settings, and seat availability checks.

6.3.2 Functional Testing

Functional testing will verify that individual features behave according to their requirements. It will focus on account creation and payment failure handling.

6.3.3 Integration Testing

Integration testing will verify that related classes and external services work together correctly. It will focus on the interaction between Seat and Policy, and between Payment, Ticket, and Seat.

6.3.4 System Testing

System testing will verify complete user workflows and overall system behavior. It will focus on successful ticket purchases, expired seat holds, and simultaneous attempts to purchase the same seat.

6.4 Test Environment and Assumptions

Testing will be performed in a designated test environment using sample customer accounts, movies, screenings, seats, and payment information.

The following assumptions apply:

- The system contains sample movies and published screenings.
- The configured seat-hold duration is five minutes.
- The maximum number of tickets per transaction is 20.
- A test payment service or test mode is available to simulate successful and failed payments.
- Tests involving time limits will use a controlled test environment.
- Actual results will be recorded after each test is executed.

6.5 Test Cases

6.5.1 Unit Testing

TestCaseId	Component	Priority	Description/Test Summary	Pre-requisites	Test Steps	Expected Result	Actual Result	Status	Test Executed By

UNIT_01	Movie	P1	Verify that searching for a movie returns matching results and that a nonexistent movie returns no matches.	Movie records exist in the system.	1. Search for an existing movie title. 2. Verify the results. 3. Search for a nonexistent title. 4. Verify the results.	The existing movie appears in the results, and the nonexistent movie returns no matching results.	The movie exists in the system.	Passed	Liliana
UNIT_02	Policy	P1	Verify that getMaxTickets() returns the maximum of 20 tickets per transaction.	The maximum ticket limit is configured as 20.	1. Access the Policy component. 2. Call getMaxTickets(). 3. Check the returned value.	The method returns 20.	Method returned 20	Passed	Shayna
UNIT_03	Seat	P1	Verify that isAvailable() correctly identifies available, temporarily held, and sold seats.	Seats can be set to all three states.	1. Set a seat to be available and check its status. 2. Set it to temporarily held and check again. 3. Set it to sold and check again.	The method identifies whether the seat is available for purchase based on its state.	Available seats displayed as available, held seats displayed as held	Passed	Thi

6.5.2 Functional Testing

TestCase Id	Component	Priority	Description/Test Summary	Pre-requisites	Test Steps	Expected Result	Actual Result	Status	Test Executed By
FUNC_01	UserInfo	P1	Verify that account creation rejects an email address that is already registered.	An account with the test email already exists.	1. Open account registration. 2. Enter the existing email address and other required information. 3. Submit the form.	The system rejects the duplicate email and does not create another account with that email.	Duplicate email was rejected	Passed	Shayna
FUNC_02	Payment	P0	Verify that a failed payment does not complete a ticket purchase.	A screening and an available seat exist. A payment failure can be simulated.	1. Select a screening and seat. 2. Begin the purchase process. 3. Submit a payment that fails. 4. Check the payment status and purchase result.	The payment is marked failed, the purchase is not completed, and no valid purchased ticket is issued.	Payment was marked failed and ticked was not issued	Passed	Thi

6.5.3 Integration Testing

TestCase Id	Component	Priority	Description/Test Summary	Pre-requisites	Test Steps	Expected Result	Actual Result	Status	Test Executed By
INT_01	Seat, Policy	P1	Verify that a seat hold expires after five minutes and the seat becomes available again.	A seat is available, and the hold duration is configured as five minutes.	1. Select an available seat. 2. Place a hold on it. 3. Wait for the five-minute hold to expire. 4. Check the seat status.	The hold expires, and the seat becomes available for another customer.	The seat was released after five minute	Passed	Liliana
INT_02	Payment, Ticket, Seat	P0	Verify that successful payment creates a ticket and marks the selected seat as sold.	A screening and an available seat exist. Successful payment can be simulated.	1. Select a screening and seat. 2. Begin the purchase process. 3. Complete payment successfully. 4. Check the payment, ticket, and seat records.	Payment succeeds, a ticket is created, and the seat is marked sold.	Payment succeeded, and the seat is marked sold	Passed	Thi
INT_03	Ticket, Policy	P1	Verify that a purchase exceeding the maximum of 20 tickets is rejected.	The maximum transaction limit is configured as 20, and sufficient seats are available for the test.	1. Begin a ticket purchase. 2. Select 21 tickets. 3. Attempt to submit the purchase.	The system rejects the transaction, and the purchase of 21 tickets is not completed.	Rejected 21 tickets	Passed	Shayna

6.5.4 System Testing

TestCase Id	Component	Priority	Description/Test Summary	Pre-requisites	Test Steps	Expected Result	Actual Result	Status	Test Executed By

SYS_01	UserInfo, Movie, Screening, Seat, Payment, Ticket	P0	Verify that a customer can complete a ticket purchase from login through confirmation.	A registered customer exists, a published screening has available seats, and successful payment is possible.	1. Log in. 2. Search for a movie. 3. Select a screening. 4. Select an available seat. 5. Complete payment. 6. Review the purchase confirmation.	The purchase completes, a ticket is created, the seat is marked sold, and confirmation is displayed.	The purchase completes, the ticket is created, the seat is marked sold and confirmatio n is displayed.	Passed	Liliana
SYS_02	Seat, Policy, Payment	P1	Verify that an unpaid seat hold expires and the seat becomes available again.	A screening has an available seat, and the hold duration is five minutes.	1. Select a seat. 2. Place a hold without completing payment. 3. Wait five minutes. 4. Check whether the seat can be selected again.	The unpaid hold expires, and the seat becomes available to another customer.	Expired held - time duration, release the seat.	Passed	Shayna
SYS_03	Seat, Ticket, Payment	P0	Verify that two customers cannot purchase the same seat for the same screening.	Two customer accounts exist, and a screening has one available test seat.	1. Have Customer A select the seat. 2. Have Customer B attempt to select the same seat while it is held. 3. Complete Customer A's purchase. 4. Check Customer B's purchase attempt.	Only one customer can purchase the seat. The system prevents the seat from being sold twice.	Only customer A successful the payment	Passed	Thi

6.6 Test Coverage

The test cases cover the main features identified in the software design.

Feature	Test Cases	Testing Method
Movie search	TC01, TC08	Unit, System
Ticket limit policy	TC02	Unit
Seat availability	TC03, TC06, TC09, TC10	Unit, Integration, System
Duplicate account prevention	TC04	Functional
Failed payment handling	TC05	Functional

Seat-hold expiration	TC06, TC09	Integration, System
Payment and ticket creation	TC07, TC08	Integration, System
Complete ticket purchase	TC08	System
Concurrent seat selection	TC10	System

The test plan includes normal operations and failure conditions. These include searches with no results, duplicate account registration, rejected payments, expired seat holds, and simultaneous attempts to purchase the same seat.

6.7 Entry and Exit Criteria

6.7.1 Entry Criteria

Testing may begin when the relevant components are available in the test environment, sample data has been prepared, and the necessary payment test service is accessible.

6.7.2 Exit Criteria

Testing is considered complete when all planned test cases have been executed, actual results have been recorded, and failures have been documented for review.

A test case will be marked as passed only when its actual result matches the expected result. Any test that has not been executed will remain marked as Not Run.

6.8 Test Deliverables

The testing deliverables will include:

- This test plan within the Software Requirements Specification.
- An Excel test-case document completed using the instructor's provided template.
- Recorded test results and documented failures after execution.
- The GitHub repository folder link containing the test plan and related project materials.

GitHub link: https://github.com/thidangg-cs/Software-Design-Specification---Group-6

7. Change Management Process

Identify and describe the process that will be used to update the SRS, as needed, when project scope or requirements change. Who can submit changes and by what means, and how will these changes be approved.

A. Appendices

Appendices may be used to provide additional (and hopefully helpful) information. If present, the SRS should explicitly state whether the information contained within an appendix is to be considered as a part of the SRS's overall set of requirements.

Example Appendices could include (initial) conceptual documents for the software project, marketing materials, minutes of meetings with the customer(s), etc.

A.1 Appendix 1

A.2 Appendix 2